

Gerrit Merge Conflict

Final Development Report

CS 5150 — Group 17

Veronica Starchenko (vs422), Vincent Huang (vh222), Luke Tao (lyt5),

Ethan Matthew Zhang (epz6), Euna Lee (ekl49)

Spring 2026

1. Progress During the Final Development Session

This section reflects on the team's work during the final sprint, comparing actual progress against the most recent project plan and identifying any risks that materialized.

1.1 Work Summary

During the final development session, the team focused on three primary areas: completing and polishing the three-panel merge UI, integrating the conflict resolution flow with Gerrit's existing inline-edit infrastructure, and preparing the codebase for handoff. The major areas of work included:

- Finalizing the gr-merge-editor web component, including simultaneous panel scrolling, word-wrap and progress toggles, and full keyboard-shortcut coverage for every resolution action.
- Wiring the save/publish path through Gerrit's existing inline-edit flow, eliminating the need for custom backend infrastructure.
- Completing the conflict parser (merge-conflict-parser) with support for both two-way and diff3 conflict marker formats.
- Hardening the undo stack and navigation logic, and verifying that conflict-marker arrows are correctly removed upon acceptance.
- Dark-mode support and tab full-width styling were added to improve visual consistency with other Gerrit features.
- Highlighting blocks of code that contain merge conflicts with different color highlights for each branch, allowing users to easily see problematic lines of code.
- Writing and passing tests achieving 95% UI coverage; running end-to-end tests against live Gerrit instances with real merge conflicts.
- Documenting all new functions and test cases to facilitate handoff.

Work proceeded largely on schedule. The team successfully merged all delivered features to the trunk branch (three-panel-merge) ahead of the final deadline, with 12 commits ahead of master representing the complete implementation.

1.2 Milestone Table

Sprint	Dates	Milestone	Status	Evidence
1	by Feb 26	Requirements, env setup, client alignment	Complete	Client meeting held; env built from source
1	by Feb 26	Frontend prototype: inline edit UI (original)	Pivoted	Upstream shipped this; redirected to merge conflict resolution
2	by Mar 28	Backend REST skeleton (GET/POST conflict endpoints)	Complete	PR #1 — commit 0aca504
2	by Mar 28	Frontend conflict resolution view + routing	Complete	PR #2 — commit 351e8f1
2	by Mar 28	REST type contracts (TS ↔ Java DTO mirror)	Complete	rest-api.ts in PR #2
3	by Apr 11	Implement merge editor UI core	Complete	gr-merge-editor.ts — commits 43f2297850, 41cdc20e1b
3	by Apr 18	Unit tests for merge editor behavior	Complete	gr-merge-editor_test.ts (“complex workflows”, undo, progress, diff3)
3	by Apr 25	UI polish + bug fixes + code freeze readiness	Complete	gr-merge-editor.ts + gr-merge-editor_test.ts — commits da60ba75f2, 45bb48cf19
4	by Apr 30	UI polish + bug fixes + testing	Complete	gr-merge-editor.ts + gr-merge-editor_test.ts — commits 61b41b5
4	by May 15	Handover Package	Complete	commit 5d0499

The final sprint plan called for completing the three-panel UI, integrating the save/publish path, achieving test coverage targets, and preparing handoff documentation. All of these milestones were met. The one area where the team fell short of original scope was the per-conflict inline button UI (discussed in Section 3), which was deliberately scoped down in agreement with the client due to visual clutter concerns.

1.3 Code Information

PR #3 (frontend, 12 TypeScript files, commit 45bb48cf19):

<https://github.com/autoelk/gerrit-merge-conflicts/pull/3>

All team members contributed to the frontend UI design, iteration, bug-fixing, and test planning/validation (pairing, design reviews, and coordinated implementation tasks). The team collaborated on:

Behavior + correctness: fixing highlights and keyboard shortcuts

Documentation: preparing for the handover package

Testing support: expanding unit tests and identifying integration/E2E test needs for the submit workflow

SWE Practices: feature branches per task → PR → peer review → merge to master (final direct push with handover package). Commit messages use Gerrit's Change-Id footer for upstream compatibility. TypeScript follows PolyGerrit's LitElement model.

1.4 Risks Manifested and Mitigation

Several risks that were identified earlier in the project manifested during the final session:

Risk	Manifestation	Mitigation
Testing complexity	E2E testing required standing up live Gerrit instances and creating real merge conflicts, which was time-consuming and slowed UI iteration.	Team set up reusable Gerrit test instances and focused unit tests on the parser and resolution logic to reduce reliance on full E2E cycles for every change.
Inline edit integration	Discovered mid-project that Gerrit already had inline-edit infrastructure suitable for the save/publish path, which changed the implementation plan.	Pivoted to reuse existing logic rather than building a custom backend, saving significant development time and reducing scope risk.
Learning curve	Polymer web components and the Bazel build system required substantial ramp-up time early in the project, compressing the implementation window.	Team front-loaded learning and pair-programmed on unfamiliar subsystems; AI tooling (Claude, Codex) accelerated boilerplate generation.
Base-column wiring	Full wiring of the base column in the three-panel view was not completed by the final session.	The feature is noted as the highest-priority follow-up item in the handoff. The UI degrades gracefully without it.

2. Feature Delivery Summary

This section provides a comprehensive account of which originally-requested features were delivered, which were revised with client approval, and which could not be completed.

2.1 Delivered Features

Three-Way Merge UI

- Fully implemented gr-merge-editor component with Current, Incoming, and editable Result panes.

- Primarily vertical layout, consistent with Gerrit's design language.
- Simultaneous scrolling across all three panels.
- Support for resolution actions: Accept Current, Accept Incoming, Accept Both (Current First), Accept Both (Incoming First), Accept All (Current), Accept All (Incoming).
- Undo action for all resolution steps.
- Navigation between conflicts within a file.

Conflict Highlighting

- Color-coded highlighting of conflict blocks, with distinct colors per branch.
- Support for both two-way and diff3 conflict marker formats.
- Active-conflict highlighting (partially implemented; noted as follow-up item).

Usability Enhancements

- Full dark-mode support.
- Tabs fill full width for improved scannability.
- Word-wrap toggle and progress indicator.
- Keyboard shortcuts available for every resolution action.

Integration with Gerrit

- Save/publish path reuses Gerrit's existing inline-edit flow, creating new patch sets without custom backend.
- API metadata fields (contains_git_conflicts, RevisionInfo.conflicts) integrated for activation and data loading.
- Reference content loaded from conflict commit IDs.

Testing

- 95% UI test coverage.
- Tests cover parser, resolution, undo, progress tracking, and fallback behavior.
- End-to-end tests run against live Gerrit instances with real merge conflicts.

2.2 Revised Features (with Client Approval)

Original Feature	Reason for Revision	Delivered Replacement	Client Approval
Per-conflict inline accept/reject popup buttons (VS Code style)	Created significant visual clutter; did not align with Gerrit's design conventions.	Accept/reject buttons at the bottom of the whole change, plus direct editing of the Result pane for fine-grained control.	Yes

Original Feature	Reason for Revision	Delivered Replacement	Client Approval
Custom backend API for merge conflict resolution	Discovered early in implementation that Gerrit's existing API provided all required data; custom backend was unnecessary.	Frontend-only implementation using Gerrit's existing API endpoints.	Yes
Backend skeleton code (written from scratch)	Existing Gerrit inline-edit logic could be ported to our use case, removing the need for new infrastructure.	Reused and extended Gerrit's inline-edit flow for the save/publish path.	Yes

2.3 Features Not Delivered

Feature	Reason Not Delivered	Status / Location
Full-file system view (multi-file conflict navigation)	Gerrit reviews on a per-file basis; a multi-file conflict navigation view would require significant changes to Gerrit's file-view architecture that were out of scope.	Not started. Would require changes to gr-editor-view and potentially Gerrit's routing layer.
Complete base-column wiring	Time constraints in the final sprint. The feature is partially scaffolded.	In progress. Located in gr-merge-editor. Remaining tasks: wire base commit data source, display in third panel, handle load failures with visible warnings.

3. Unfinished Work in Progress

The following items are in progress or partially implemented and were not completed by the final session. Future developers should consult the three-panel-merge branch and the GitHub Issues tracker for detailed context.

3.1 Base-Column Wiring

- Location: gr-merge-editor component, three-panel-merge branch.
- Status: Partially scaffolded. The UI renders three panels but the base (common ancestor) column is not fully wired to its data source.
- Remaining tasks: Connect base commit ID to reference-pane data loader; handle failed reference-pane loads with a visible warning message to the user; validate display against real three-way conflicts.

3.2 Active-Conflict Highlighting Polish

- Location: gr-merge-editor, merge-conflict-parser.
- Status: Basic highlighting implemented. Active-conflict visual emphasis and scroll-to-active behavior are incomplete.
- Remaining tasks: Implement scroll-to-active-conflict on navigation; add stronger visual emphasis (border or background) to the currently focused conflict block.

3.3 CodeMirror Integration

- Location: gr-merge-editor Result pane.
- Status: Not started. Current implementation uses a plain textarea.
- Remaining tasks: Evaluate and integrate CodeMirror for syntax highlighting and a richer editing experience in the Result pane.

3.4 Re-parse-after-Mutation

- Location: merge-conflict-parser, gr-merge-editor.
- Status: The parser re-parses on initial load but does not re-parse after the user edits the Result pane directly.
- Remaining tasks: Preserve and trigger re-parse-after-mutation to keep conflict state consistent after direct edits.

4. User Testing and Feedback

4.1 Methodology

The team conducted a user testing session with 3 participants, all graduate students in computer science with regular Git experience but no prior Gerrit exposure. Participants were recruited from outside the course and were not team members or the client. Each session lasted approximately 25–30 minutes and was conducted in person. A team member observed and took notes while another operated a shared laptop running a live Gerrit instance pre-loaded with a sample repository containing staged merge conflicts. Participants were asked to think aloud as they worked. A short 5-question survey (1–5 Likert scale) was administered at the end of each session.

4.2 Tasks

Participants were asked to complete the following tasks, each with a time limit. Tasks were presented sequentially; participants were given a brief orientation to Gerrit's general interface before beginning but received no explanation of the merge conflict feature.

#	Task Description	Time Limit	Use Case Covered
1	Given a Gerrit change with a merge conflict, navigate to the merge conflict resolution view and identify all conflicting regions.	5 min	Initiating conflict resolution

#	Task Description	Time Limit	Use Case Covered
2	Resolve a merge conflict by accepting the incoming change for one conflict and editing the result manually for another, then save the result.	8 min	Resolving conflicts and saving
3	Use the undo feature to revert a resolution decision, then re-resolve using a different option.	5 min	Undo and re-resolution
4	Toggle dark mode and word wrap, then navigate between conflicts using keyboard shortcuts only.	5 min	Usability and accessibility

4.3 Results

All three participants completed Tasks 1–3 within the time limit. Task 4 proved more challenging: two of three participants successfully toggled dark mode and word wrap but required a hint to discover the keyboard shortcut reference. Individual participant timing is shown below.

Task	P1 Time	P2 Time	P3 Time	Notable Observations
1	1 min 42 sec	2 min 15 sec	1 min 58 sec	All participants located the conflict view without difficulty. P2 initially clicked the diff view before finding the merge editor tab.
2	5 min 10 sec	6 min 44 sec	4 min 52 sec	P1 and P3 completed smoothly. P2 spent extra time on the manual edit step, unsure whether changes were being saved incrementally or only on explicit save.
3	2 min 5 sec	2 min 38 sec	1 min 55 sec	All participants found the undo button quickly. P2 noted they expected Ctrl+Z to work in the Result pane.
4	3 min 20 sec	Did not complete	4 min 10 sec	P1 and P3 found dark mode and word wrap toggles but needed prompting to discover keyboard shortcuts. P2 was unable to complete keyboard-only navigation within the time limit.

4.4 Survey Results

Participants completed a 5-question post-session survey using a 1–5 Likert scale (1 = strongly disagree, 5 = strongly agree). Average scores across all three participants are shown below.

Survey Question	Avg. Score	Range
I was able to understand which lines of code were in conflict.	4.7	4–5
The resolution workflow (accept/edit/save) was intuitive.	4.0	3–5
The color highlighting helped me identify conflicting regions quickly.	4.3	4–5
I would find this tool useful in my own development workflow.	4.3	4–5
The keyboard shortcuts were easy to discover and use.	2.7	2–3

Qualitative feedback themes: participants appreciated the side-by-side panel layout and found it more intuitive than manually editing conflict markers in a text editor. The most consistent criticism was that keyboard shortcuts were not sufficiently discoverable since participants expected a visible shortcut reference or tooltip rather than a separate menu. One participant commented that the Result pane “felt basic” compared to the read-only panels and expressed a preference for syntax highlighting there.

4.5 Issues Identified and Suggestions for Improvement

- Keyboard shortcut discoverability: Two of three participants required prompting to find the keyboard shortcut reference. A persistent shortcut legend or contextual tooltip on the navigation buttons would reduce this friction.
- Save state clarity: One participant was uncertain whether edits to the Result pane were auto-saved or required an explicit action. A clearer save-state indicator (e.g., “Unsaved changes” / “Saved”) adjacent to the save button would address this.
- Ctrl+Z expectation in Result pane: One participant expected Ctrl+Z to trigger undo within the Result pane. The current undo button covers conflict-level undo but does not intercept the browser’s native undo in the textarea; aligning these behaviors would reduce confusion.
- Syntax highlighting in Result pane: One participant noted the editable Result pane felt visually inconsistent with the read-only Current and Incoming panes. This supports the planned CodeMirror integration as a meaningful usability improvement.
- Initial tab discoverability: One participant briefly clicked the standard diff tab before finding the merge editor tab. A more prominent label or badge on the merge editor entry point may help first-time users orient more quickly.

5. Presentation and Acceptance Testing Takeaways

The final client presentation covered the full merge conflict resolution workflow via live demo. The overall reception was positive, with the client satisfied with the scope and quality of delivered features. The client’s primary feedback was focused on testing:

- The client’s main recommendation was to improve end-to-end and integration test coverage before the feature is considered ready for broader integration into the Gerrit codebase. While the team achieved 95% UI unit test coverage, the client noted that validating the full workflow against a wider variety of real merge conflict scenarios —

including edge cases such as overlapping edits, adjacent conflicts, and large files — would be necessary to build confidence for upstream submission.

- The client specifically suggested ensuring that the save/publish path (via Gerrit's inline-edit flow) is exercised end-to-end in automated tests, so that future contributors can reproduce results and catch regressions without needing to stand up a full Gerrit instance manually each time.
- No major design or feature concerns were raised. The three-panel vertical layout and the decision to reuse Gerrit's inline-edit infrastructure were accepted without objection.

Strengthening E2E and integration test coverage is therefore the highest-priority remaining task and is reflected as such in the handoff package.

6. Handoff Package Index

The handoff package consists of this report, the source code on the three-panel-merge branch, and the supporting documents listed below. Documents are delivered as standalone files alongside this report. The index notes which items still require manual completion by the team.

Deliverable	Status	Notes
This report	Complete	Submitted via course portal.
Source code	Complete: merged to trunk from three-panel-merge branch	Key components at polygerrit-ui/app/elements/: gr-merge-editor, gr-editor-view, merge-conflict-parser.
User's Manual	Complete: delivered as separate document	See Gerrit MergeConflicts UserManual.docx .
Maintainer's Manual	Complete: delivered as separate document	See Gerrit MergeConflicts MaintainerManual.docx .
Demo video walkthrough	Complete: delivered on Youtube	See https://www.youtube.com/watch?v=ZQ9Iz7CSBS E 5–10 minute screen recording covering: navigating to the merge editor, resolving conflicts (accept/edit/save), undo, and keyboard shortcuts.
License agreement	Complete: signed and delivered	Confirmed agreement to publish under Apache 2.0 (consistent with Gerrit's license). Committed a signed LICENSE and CONTRIBUTORS file to the repository root.

7. Conclusion

The team successfully delivered a functional, Gerrit-native three-panel merge conflict resolution UI. The core value of the project, allowing developers to resolve merge conflicts directly in the browser within Gerrit's familiar interface, was achieved. By reusing Gerrit's existing inline-edit infrastructure and design patterns, the implementation minimizes technical debt and is positioned well for integration into the broader Gerrit codebase.

Key engineering decisions that paid off: frontend-only architecture, reuse of existing inline-edit flow, and a focused per-file approach consistent with Gerrit's design philosophy. The main areas requiring follow-up are base-column wiring, active-conflict highlighting polish, and CodeMirror integration which are all clearly documented in the handoff package for future contributors.